A BIDIRECTIONAL DEEP LEARNING FRAMEWORK FOR REAL-TIME SIGN LANGUAGE TO TEXT AND TEXT TO SIGN TRANSLATION

Project Phase-I Report submitted to Osmania University, Hyderabad, in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF ENGINEERING

in

INFORMATION TECHNOLOGY

By

Mr. Md Bilal Uddin (160923737027)
Mr. Mohd Safwaan (160923737034)
Mr. Mohammed Ubaid Ur Rahman Siddiqui (160923737041)

Under the Guidance of
Mrs. B Swaroopa
Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY
LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayath sagar, Hyderabad,500091

A.Y. 2026-2027



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayath sagar, Hyderabad, 500091

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the major project report on **"A BIDIRECTIONAL DEEP LEARNING FRAMEWORK FOR REAL-TIME SIGN LANGUAGE TO TEXT AND TEXT TO SIGN TRANSLATION"** is a bonafide work carried out by **Mr. Md Bilal Uddin (160923737027), Mr. Mohd Safwaan (160923737034), and Mr. Mohammed Ubaid Ur Rahman Siddiqui (160923737041)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Information Technology is a record of bonafide work carried out by them.

Internal Project Guide
Mrs. B Swaroopa
Assistant Professor

Dr. K. Nagi Reddy
H.O.D, I.T

External Examiner



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayath sagar, Hyderabad, 500091

DEPARTMENT OF INFORMATION TECHNOLOGY

DECLARATION BY THE CANDIDATES

We, **Mr. Md Bilal Uddin (160923737027), Mr. Mohd Safwaan (160923737034), and Mr. Mohammed Ubaid Ur Rahman Siddiqui (160923737041)**, hereby declare that the major project report entitled "**A BIDIRECTIONAL DEEP LEARNING FRAMEWORK FOR REAL-TIME SIGN LANGUAGE TO TEXT AND TEXT TO SIGN TRANSLATION**" is carried out under the guidance of **Mrs. B Swaroopa**, Assistant Professor, Department of Information Technology, **Lords Institute of Engineering and Technology, Hyderabad,** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Information Technology.**

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Md Bilal Uddin (160923737027)
Mohd Safwaan (160923737034)
Mohammed Ubaid Ur Rahman Siddiqui (160923737041)

ACKNOWLEDGEMENT

We are very pleased to present this thesis of our project. This period of our student life has been truly rewarding. A number of people were of immense help to us during the course of our research and the preparation of our thesis.

First, we wish to thank **GOD Almighty** who created heavens and earth, who helped us in completing this project and we also thank our parents who encouraged us in this period of research.

We would like to express our sincere gratitude to **The Management** of **Lords Institute of Engineering and Technology, Hyderabad** for providing us with an excellent platform and resources to successfully carry out this project.

We would also like to thank **Dr. Ravi Kishore Singh, Principal, L.I.E.T.** for extending his help in overall completion of this project work.

We would like to express our deep sense of gratitude **to Dr. K. Nagi Reddy, Professor & HOD**, **Information Technology, Lords Institute of Engineering & Technology, Hyderabad,** for his encouragement and cooperation throughout the project.

We would like to thank **Mrs. B Swaroopa, Assistant Professor, Dept. of Information Technology, Lords Institute of Engineering & Technology**, Hyderabad, our project internal guide, for her guidance and help. Her insight during the course of our research and regular guidance were invaluable to us.

Lastly, we acknowledge the support of our faculty members, friends, and family, who inspired and encouraged us throughout this journey.

Md Bilal Uddin (160923737027)
Mohd Safwaan (160923737034)
Mohammed Ubaid Ur Rahman Siddiqui (160923737041)

Vision of the Institute

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

PRINCIPAL
Principal
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Department:

To be a frontier in Information Technology focusing on quality professional education, innovation and employability.

Mission of the Department:

DM1: Adopt innovative teaching learning processes to achieve stated outcomes and sustained performance levels.

DM2: Provide learner centric environment to enrich knowledge, skills and innovation through industry academia collaborations.

DM3: Train the concerned stakeholders on latest technologies to meet the industry needs.

DM4: Inculcate leadership, professional, interpersonal skills and ethical values to address the contemporary societal issues.

Note: DM: Department Mission

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E Information Technology Program Educational Objectives (PEOs):

PEO1	Be a competent software engineer in IT and allied industry providing viable solutions.
PEO2	Exhibit professionalism, engage in lifelong learning and adapt to changing industry and societal needs.
PEO3	Be effective as an individual and also cohesive in a group to execute multidisciplinary projects.
PEO4	Pursue higher studies in Information Technology with emphasis on competency and innovation.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2002.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E Information Technology Program Specific Outcomes (PSOs):

PSO1	**Professional Skills:** Identify, analyze requirements, design and implement computer programs in the area of artificial intelligence, machine learning, big data analytics and networking of varying complexity.
PSO2	**Problem-Solving Skills:** Deliver quality service to IT industry and provide solutions to real-life problems applying modern computer languages, software methods and tools.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

Course Outcomes: Project Phase- I

After completion of the project, the student will be able to

S.No	Description	Blooms Taxonomy Level
PP.I.CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to the real-world problems.	BTL3
PP.I.CO2	Evaluate different solutions based on economic and technical feasibility	BTL4
PP.I.CO3	Effectively plan a project and confidently perform all aspects of project management.	BTL5
PP.I.CO4	Demonstrate effective written and oral communication skills	BTL4
PP.I.CO5	Prepare the documentation report and perform the presentation of the project work	BTL3

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2002.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Articulation Matrix:

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

Course Outcomes (COs)	Program Outcomes (POs)										Program Specific Outcomes (PSOs)		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO 11	PSO 1	PSO2
PP.I.CO1	3	2	2	-	2	2	-	-	3	-	-	2	2
PP.I.CO2	3	3	2	3	2	-	2	-	3	-	-	-	2
PP.I.CO3	3	3	3	-	3	3	1	2	2	3	2	3	-
PP.I.CO4	3	2	2	3	3	3	3	2	-	-	2	3	3
PP.I.CO5	3	3	3	3	3	3	3	2	3	3	-	3	3
AVG	**3**	**2.6**	**2.5**	**3**	**2.6**	**2.7**	**2.2**	**2**	**2.9**	**3**	**2**	**2.7**	**2.5**

Level:

1-Low correlation (Low), 2- Medium correlation (Medium), 3-Highcorrelation (High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING		9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	✓	15 LIFE ON LAND	
4 QUALITY EDUCATION	✓	10 REDUCED INEQUALITIES	✓	16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES		17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

Sign language serves as a crucial medium of communication for deaf and hard-of-hearing individuals. However, communication between sign language users and non-signers remains challenging due to the limited understanding of sign language among the general population. To address this communication gap, this project proposes **SignNet II, a bidirectional deep learning framework for real-time Sign Language to Text and Text to Sign Translation**. The proposed system is designed to perform translation in both directions, converting sign gestures into textual words and translating the predicted text back into corresponding sign images.

The proposed architecture consists of two Transformer-based attention encoder-decoder models. The first model processes sign gestures captured through video or a live webcam and generates the corresponding textual output. The second model takes the predicted text as input and generates the corresponding sign representation. The **OpenPose** algorithm is used to extract meaningful hand and finger key-points from video frames, while **Word2Vec** is employed to represent textual data in numerical form for efficient processing. The system uses a word-level **American Sign Language (ASL) dataset containing 18 distinct words** and is implemented through a **Tkinter-based graphical user interface** that supports both uploaded video and real-time webcam input.

The performance of the proposed system is evaluated using the **BLEU score**, which measures the similarity between the predicted text and the corresponding ground-truth text. Overall, SignNet II provides an interactive two-way translation framework that aims to improve accessibility, facilitate communication, and provide a practical solution for real-time sign language interaction.

Keywords: Sign Language Translation, SignNet II, Transformer, OpenPose, Word2Vec, BLEU Score, Encoder-Decoder, American Sign Language, Gesture Recognition, Real-Time Translation, Two-Way Translation, Deep Learning, Tkinter GUI.

CONTENTS

Certificate i

Declaration ii

Acknowledgement iii

Vision and Mission of the Institute iv

Vision and Mission of the Department v

Program Educational Objectives vi

Program Outcomes vii

Program Specific Outcomes viii

Course Outcomes ix

Course Articulation Matrix x-xi

Abstract xii-xiii

CHAPTER 1 – INTRODUCTION 1

1.1 Background **1**

1.2 Problem Statement **2**

1.3 Aim **2**

1.4 Objectives **2**

1.5 Scope of the Project **3**

1.6 Significance **3**

1.7 Organization of the Report **4**

CHAPTER 2 – LITERATURE SURVEY 5

2.1 Review of Existing Work **5**

CHAPTER 3 – SYSTEM ANALYSIS AND REQUIREMENTS 8

3.1 Introduction **8**

3.2 Problem Analysis **8**

3.3 Proposed System **9**

3.4 Objectives 9

3.5 Stakeholders and Roles 9

3.6 Functional Requirements 10

3.7 Risk Categories 11

3.8 Risk-Scoring Rule 11

3.9 Non-Functional Considerations 12

3.10 Summary 12

CHAPTER 4 – SYSTEM STUDY AND DESIGN 13

4.1 Introduction 13

4.2 Architectural Design 13

4.3 Main Modules 14

4.4 Core Workflow 16

4.5 Logical Data Design 18

4.6 Risk Assessment and Remediation Flow 19

4.7 Security and Deployment Options 20

4.8 Summary 20

CHAPTER 5 – INPUT AND OUTPUT DESIGN 21

5.1 Introduction 21

5.2 Input Design 21

5.3 Input Validation 21

5.4 Output Design 22

5.5 Output Integrity 23

5.6 Summary 23

CHAPTER 6 – SOFTWARE ENVIRONMENT **24**

6.1 Introduction **24**

6.2 Frontend Environment **24**

6.3 Backend Environment **24**

6.4 AI and Machine-Learning Environment **24**

6.5 Database and Optional Vector Layer **25**

6.6 Security and DevOps **25**

6.7 Testing Tools **25**

6.8 Summary **25**

CHAPTER 7 – SYSTEM TESTING & RESULTS DISCUSSION **26**

7.1 Introduction **26**

7.2 Testing Objectives **26**

7.3 Testing Methods and Tools **26**

7.4 Representative Test Cases **27**

7.5 Evaluation Metrics **29**

7.6 Results and Evidence **30**

7.7 Limitations **31**

7.8 Summary **32**

CONCLUSION **33**

FUTURE SCOPE **34**

REFERENCES **35**

List of Figures

S. No.	Figure No.	Name of the Figure	Page No.
1	4.1	SignNet II System Architecture	13
2	4.2	SignNet II Class Diagram	16
3	4.3	User Interaction Sequence Diagram	17
4	4.4	SignNet II Data Flow Diagram	18
5	7.1	SignNet II Main Graphical User Interface	28
6	7.2	Dataset Loading Status and Class Distribution Graph	28
7	7.3	Real-Time Webcam Sign-to-Text and Text-to-Sign Translation	29

List of Tables

S. No.	Table No.	Name of the Table	Page No.
1	2.1	Literature Survey of Existing Sign Language Translation Methods	5
2	3.1	Stakeholders and Their Roles	9
3	3.2	Risk Categories and Their Possible Effects	11
4	3.3	Risk Scoring and Priority Levels	12
5	4.1	System Architecture Components and Responsibilities	14
6	4.2	Logical Data Groups and Their Contents	18
7	4.3	Risk Assessment and Remediation	19
8	6.1	AI and Machine-Learning Technologies Used	24
9	7.1	Representative System Test Cases	27

CHAPTER 1
INTRODUCTION

1.1 Background

Sign language is an important means of communication for Deaf and Hard of Hearing (DHH) individuals. It uses visual and spatial information, including hand shapes, finger positions and movements, to communicate meaning. Communication can become difficult when a person who uses sign language interacts with someone who does not understand it. This creates a need for accessible systems that can help interpret sign language.

Sign languages are not simply visual versions of spoken languages; they have their own vocabulary, grammatical structures and communication patterns. Understanding a sign requires analysing multiple visual features, such as hand orientation, finger configuration, movement direction and spatial positioning. Therefore, developing an automated sign language translation system requires techniques capable of extracting and interpreting these visual features accurately. Computer vision helps identify relevant information from images and videos, while deep learning enables systems to learn patterns from training data and use them for gesture classification and prediction.

Recent developments in artificial intelligence, computer vision and deep learning have made automated sign language translation a practical research area. Many existing systems concentrate on converting signs into text. However, communication also requires the reverse direction: converting text into a sign representation that can be understood visually.

The project, titled “A Bidirectional Deep Learning Framework for Real-Time Sign Language to Text and Text to Sign Translation”, presents SignNet II. It is designed as a two-way translation system. The first direction analyses sign gestures from a video or live webcam and predicts a text word. The second direction uses the predicted text to produce the corresponding sign image. The project uses a word-level American Sign Language (ASL) dataset containing 18 distinct words.

The documented design describes Transformer attention-based encoder-decoder models, OpenPose-based hand and finger key-point extraction, and Word2Vec text representation. The application is presented through a graphical user interface developed with Python Tkinter. The implementation excerpts in the source document also show MediaPipe Hands for landmark detection and TF-IDF vectorization for text preprocessing; these implementation details are recorded separately in Chapter 6.

1.2 Problem Statement

Most conventional sign language translation applications focus mainly on sign-to-text conversion. Such one-way systems provide limited support for two-way interaction because they do not provide a corresponding sign representation after text is produced or supplied. Recognition may also be affected by inaccurate hand-feature extraction, limited word coverage and difficulty processing live video.

The problem addressed by this project is the development of an interactive system that can recognise a defined set of ASL hand signs from uploaded video and webcam input, convert the recognised signs into text, and generate a corresponding sign image from the predicted text. The system must connect these operations through a user-friendly interface and provide an evaluation of its prediction performance.

1.3 Aim

The aim of the project is to design and implement SignNet II, a bidirectional deep-learning-based sign language translation framework that converts hand gestures into text and converts the resulting text back into corresponding sign images, with support for uploaded video and live webcam input.

1.4 Objectives

Develop a system for recognising sign language gestures from video files and realtime webcam input.

Build a two-way translation workflow covering sign-to-text and text-to-sign conversion.

Use an encoder-decoder learning architecture with attention mechanisms for sequence And feature processing.Extract meaningful hand and finger key-points from visual input for sign representation.Convert text labels into numerical representations suitable for model processing.

Prepare and preprocess the dataset, including shuffling, normalisation and an 80:20 training/testing split.

Integrate the processing and prediction modules into a Tkinter graphical user interface.

Evaluate sign-to-text prediction against reference labels using the BLEU score.

1.5 Scope of the Project

The project scope is limited to a word-level ASL dataset containing 18 sign classes: I, apple, can, get, good, have, help, how, like, love, my, no, sorry, thank-you, want, yes, you and your. The source document reports a dataset containing 527 images across these 18 classes.

The functional scope also includes two major operational components. The first component focuses on visual sign recognition, in which hand gesture information is captured through video or webcam input and processed to generate a corresponding text prediction. The second component focuses on sign-image generation, where the corresponding visual representation is displayed based on the selected or predicted text word. Both components are integrated through a common graphical interface to provide a unified user experience.

The application supports dataset loading, preprocessing, train-test splitting, model loading/training workflow, video-based prediction and webcam-based prediction. For a recognised sign, the system displays the predicted text and the corresponding sign image. The project is intended as a focused prototype for the specified vocabulary and should not be interpreted as a general-purpose translator for every sign language, signer or continuous sentence.

The source describes real-time webcam interaction, but also notes that correct predictions depend on presenting the hand sign accurately according to the dataset. Continuous long-running prediction is discouraged in the source because the application is computationally heavy.

1.6 Significance

The project demonstrates how computer vision and deep learning can be combined to support communication between sign-language users and people who do not know sign language. Its two-way design is significant because it provides both a text interpretation of a sign and a visual sign representation corresponding to text.

The system also demonstrates an end-to-end workflow involving dataset preparation, hand landmark extraction, model inference, text output and image output. The Tkinter interface makes these functions accessible through selectable operations rather than requiring users to interact directly with model code. The project can serve as a foundation for further work on larger vocabularies, more diverse signers and continuous sign-language translation.

In addition to its technical contribution, the project highlights the importance of inclusive technology in modern society. A system that supports both sign recognition and visual sign representation can help reduce communication barriers in educational environments, demonstrations and basic everyday interactions. It also provides an opportunity to understand how different artificial intelligence techniques, including computer vision, feature extraction and deep learning, can be integrated into a practical software application. Although the current prototype has a limited vocabulary, its modular design provides a foundation for extending the system with additional sign classes and improved translation capabilities.

1.7 Organization of the Report

This report is organised into six chapters. Chapter 1 introduces the project, its problem, aim, objectives, scope and significance. Chapter 3 analyses the problem and describes the proposed system, requirements, stakeholders and project risks. Chapter 4 explains the system architecture, modules, workflow, logical data design and risk-handling flow. Chapter 5 describes the input and output design and the validation considerations. Chapter 6 presents the software and hardware environment, libraries and development tools. Chapter 7 discusses the testing approach, representative test cases, evaluation metric, documented results and limitations.

CHAPTER 2
LITERATURE SURVEY

2.1 Review of Existing Work

The development of automatic Sign Language Translation has attracted significant attention with the advancement of Deep Learning, Computer Vision, and Neural Machine Translation techniques. Several researchers have proposed systems for recognizing continuous sign language gestures and translating them into spoken or written language. The following studies provide the foundation for the development of the proposed SignNet II system.

Table 2.1: Literature Survey

SNO	Title of Paper	Methodology	Results	Advantages	Drawbacks
1	**Neural Sign Language Translation (2026)**	Uses Neural Machine Translation to learn spatial representations, language models, and the mapping between sign language videos and spoken language. The work introduces the RWTH-PHOENIX-Weather 2014T1 continuous sign language translation dataset.	The reported upper bound was 19.26 BLEU-4, while the end-to-end frame-level and gloss-level models achieved 9.58 and 18.13 BLEU-4 respectively.	Provides an important foundation for continuous Sign Language Translation and considers the linguistic structure and different word orders of sign language.	The described translation task focuses on converting sign language videos into spoken language and does not provide a two-way Sign-to-Text and Text-to-Sign communication process.

2	**Large Scale Sign Language Interpretation (2025)**	Uses sequence-to-sequence deep learning approaches for translating Chinese Sign Language into English and Mandarin written text. The approach utilizes body joint position, facial expression, and finger articulation.	Introduces a large dataset containing 50,000 video snippets from 10,000 unique utterances and 50 signers. The study demonstrates the potential of deep learning for large-scale sign language interpretation.	Uses multiple visual features and a large-scale dataset, providing a broader basis for automatic sign language interpretation.	The study focuses on Chinese Sign Language to written-language translation, while generalization to unseen utterances remains challenging.
3	**Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation (2023)**	Proposes a Transformer-based architecture that jointly performs continuous sign language recognition and translation. A CTC loss is used to connect recognition and translation within a unified end-to-end architecture.	Evaluated on the RWTH-PHOENIX-Weather-2014T dataset. The reported translation performance reached 21.80 BLEU-4, improving over earlier reported results of 9.58 BLEU-4.	Demonstrates the effectiveness of Transformer architectures and joint recognition-translation learning for sign language processing.	The work primarily addresses continuous sign language recognition and translation and requires a large sentence-level dataset, rather than focusing on a small word-level real-time two-way translation system.

The reviewed studies demonstrate that deep learning and Transformer-based approaches can effectively learn the relationship between visual sign language sequences and natural language. Neural Sign Language Translation established the use of neural machine translation for continuous sign language, while Large Scale Sign Language Interpretation demonstrated the usefulness of body, facial, and finger features for large-scale translation. Transformer-based research further improved translation performance by jointly learning recognition and translation.

Based on these studies, the proposed **SignNet II** focuses on addressing the limitations identified in existing approaches by providing a bidirectional translation framework. The proposed system performs both Sign-to-Text and Text-to-Sign translation, uses OpenPose for hand and finger key-point extraction, employs Word2Vec for textual representation, and supports video as well as real-time webcam input. The system uses a word-level ASL dataset containing 18 distinct words and evaluates the translation performance using the BLEU score.

Comparative Analysis of Existing Literature

The literature survey table presents a comparative analysis of existing sign language translation systems and the techniques used in their development. It highlights the approaches, advantages, and limitations of previous research, identifying gaps such as one-way translation, dataset limitations, gesture-recognition challenges, and insufficient real-time support. This comparison provides the foundation for developing SignNet II, which aims to overcome these limitations by integrating sign recognition and sign-image generation into a single application.

The literature survey table provides a detailed comparative analysis of various existing research papers and systems related to sign language translation. The comparison helps identify major challenges in existing systems, including one-way translation, limited dataset flexibility, difficulties in accurate hand gesture recognition, lack of real-time processing, and the absence of visual sign generation.. Based on these findings, SignNet II is proposed to integrate sign-to-text recognition and text-to-sign image generation into a single application, providing a more complete communication support system.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Introduction

System analysis identifies the limitations of the existing approach and defines the functions required in the proposed application. SignNet II is intended to provide a complete two-way workflow using a defined ASL vocabulary. This chapter presents the problem analysis, proposed solution, users, functional requirements, risk categories and non-functional considerations.

3.2 Problem Analysis

The project source identifies several limitations in existing sign language translation systems, which are described below:

1. **One-Way Translation:** Many existing systems convert sign language gestures into text but do not support the reverse process of converting text into corresponding sign representations.
2. **Dataset Constraints:** Some systems have limited flexibility when working with word-level sign language datasets, restricting their ability to recognise a wider range of signs.
3. **Gesture-Recognition Difficulty:** Identifying detailed hand shapes, finger positions and movements accurately remains a challenge, which may affect the overall recognition performance.
4. **Limited Real-Time Support:** Some systems do not adequately support live webcam input or continuous video stream processing, limiting their usefulness in interactive communication.
5. **Absence of Visual Feedback:** The recognised text may not be regenerated into its corresponding sign image, making it difficult for users to understand the translated output visually.

These limitations reduce the usefulness of a translation system as an interactive communication aid. SignNet II addresses them by joining sign recognition and sign-image generation in one application. Its defined vocabulary and visual input requirements also establish the operating boundary of the prototype.

3.3 Proposed System

The proposed system, SignNet II, uses two connected translation directions. In the sign-to-text direction, a video frame or webcam frame is processed to identify hand landmarks. The extracted features are preprocessed and supplied to the recognition model, which predicts one of the supported words. In the text-to-sign direction, the predicted word is represented numerically and passed through the image-generation/reconstruction path to obtain its corresponding sign image.

The project documentation describes Transformer attention-based encoder-decoder models and a dual-learning approach. It identifies OpenPose for hand and finger key-point extraction and Word2Vec for text encoding. The code excerpts further document MediaPipe Hands, a custom attention layer, CNN/LSTM components and TF-IDF vectorization. These distinctions are important when describing the design concept separately from the visible implementation.

The user interacts with the application through a Tkinter GUI. The interface includes controls for uploading the dataset, preprocessing it, splitting the data, training or loading the SignNet II models, predicting from a video and predicting from a webcam.

3.4 Objectives

Provide a two-way sign language translation process.
Recognise the 18 supported ASL word classes from visual hand-sign input.
Process hand landmark information and prepare it for model inference.
Translate predicted text into a corresponding sign image.
Support both uploaded test videos and live webcam input.
Present prediction output through the graphical interface and evaluate sign-to-text output using BLEU.

3.5 Stakeholders and Roles

The system involves several stakeholders who contribute to its development, operation and evaluation. The Application User selects the dataset or input mode, provides video

or webcam-based sign gestures, and views the predicted text and corresponding sign images. The Dataset Provider/Maintainer supplies and organises the labelled sign images required for model preparation and training. The Project Developer is responsible for implementing the graphical user interface, preprocessing pipeline, model integration, prediction functions and testing procedures. The Project Evaluator/Guide reviews the project objectives, system workflow and reported evaluation results to ensure that the implementation meets the defined requirements. Finally, Deaf and Hard of Hearing (DHH) Users represent the intended beneficiary group who may use the system as an assistive communication prototype, subject to its limited vocabulary and gesture-recognition capabilities.

3.6 Functional Requirements

FR-01 Dataset selection: the application shall allow the user to select the dataset folder.

FR-02 Dataset loading: the application shall read the available labelled sign images and associated word labels.

FR-03 Feature preparation: the application shall extract or load hand-feature data and prepare text and sign data for processing.

FR-04 Preprocessing: the application shall shuffle and normalise the processed data.

FR-05 Data splitting: the application shall divide the dataset into 80% training data and 20% testing data.

FR-06 Model operation: the application shall train or load the required SignNet II model components.

FR-07 Video prediction: the application shall allow a user to select a test video and process its frames.

FR-08 Webcam prediction: the application shall capture frames from the live webcam for sign recognition.

FR-09 Text display: the application shall display the predicted word in the GUI and/or video output.

3.7 Risk Categories

The following risk categories are relevant to the documented SignNet II workflow. The source project does not provide a formal risk register, so this section classifies practical risks for project analysis rather than reporting measured incidents.

Risk category	Project-specific risk	Possible effect
Dataset risk	Missing, incorrectly labelled or unevenly represented sign images.	Poor learning or incorrect class prediction.
Input quality risk	Hand is not visible, sign is performed inaccurately, or lighting/background affects detection.	Landmarks may not be detected reliably.
Model risk	Model weights are unavailable, incompatible or do not load correctly.	Training or prediction cannot proceed.
Integration risk	GUI, camera, preprocessing and model components do not work together.	Workflow interruption or missing output.
Evaluation risk	BLEU score is treated as a complete measure of all system behaviour.	Recognition and image-generation quality may be misinterpreted.
Operational risk	Camera or video file cannot be accessed.	Live or uploaded-video prediction fails.

3.8 Risk-Scoring Rule

For consistent prioritisation, a simple proposed risk-scoring rule may be used. This rule is an analysis aid and is not a score reported by the source project.

Risk Score = Likelihood × Impact. Each factor is rated from 1 (low) to 5 (high). The resulting score ranges from 1 to 25.

Score range	Priority	Suggested response
1–5	Low	Monitor during normal testing.
6–12	Medium	Add a validation or recovery step and retest.
13–19	High	Address before demonstration or operational use.
20–25	Critical	Do not proceed until the risk is reduced.

Example: if webcam access failure is assessed as likelihood 3 and impact 4, its score is 12 (Medium). The project team should verify camera availability before starting webcam prediction and provide a clear message if the camera cannot be opened. Actual likelihood and impact values should be assigned by the project team after testing.

3.9 Non-Functional Considerations

Performance: video and webcam processing should complete at a practical speed on the available hardware. The source notes that the project is computationally heavy.

Reliability: dataset loading, model loading, prediction and camera release should be Handled consistently.

Compatibility: the documented environment is Windows 10, Python, an Intel i3 processor, 1.1 GHz speed, 4 GB RAM and 500 GB hard disk.

Maintainability: the workflow is divided into separate dataset, preprocessing, splitting, training and prediction functions.

3.10 Summary

This chapter analysed the limitations addressed by SignNet II and described the proposed two-way translation system. It identified the main users, functional requirements and project-specific risks. It also established non-functional considerations, particularly usability, processing load, reliability and the limitations imposed by the defined dataset.

CHAPTER 4

SYSTEM STUDY AND DESIGN

4.1 Introduction

System design explains how the required functions are organised and how data moves through the application. SignNet II is structured as a desktop GUI application connected to dataset-processing functions, hand-feature extraction, sign-to-text prediction and text-to-sign image generation. The two translation directions are connected through the predicted word.

4.2 Architectural Design

The system follows a modular encoder-decoder design. The input layer accepts either a selected video file or frames from a webcam. The hand-processing stage detects hand landmarks and prepares the landmark coordinates. The sign-to-text model uses these features to predict a word from the supported classes. The predicted word is then passed to the text-to-sign path, which produces the corresponding sign image. The Tkinter GUI coordinates user actions and displays the output.

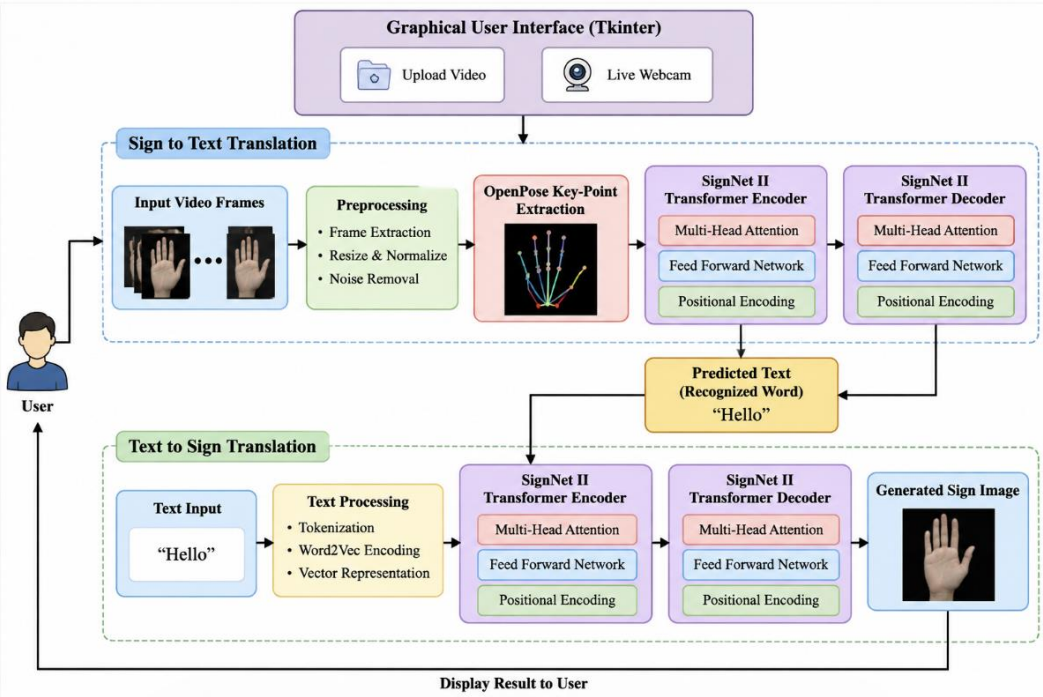


Figure 4.1: SignNet II System Architecture.

The project document describes the conceptual architecture as two Transformer attention-based encoder-decoder models. It identifies OpenPose as the key-point

extraction method and Word2Vec as the text representation method. The implementation code shown in the same source uses MediaPipe Hands for landmark extraction and includes a custom attention layer with CNN/LSTM components in the encoder, while the text preprocessing code uses TF-IDF. The report therefore distinguishes the stated architecture from the concrete libraries and code structures visible in the implementation.

Layer/component	Responsibility
Tkinter GUI	Provides dataset, preprocessing, split, model, video and webcam controls; displays textual output.
Input acquisition	Reads images from the dataset, frames from a selected video, or frames from the webcam.
Hand feature extraction	Detects hand landmarks, calculates coordinates and prepares a normalised feature vector.
Sign-to-text model	Processes hand features and predicts a word label.
Text representation	Converts the predicted word into a numerical representation for the reverse translation path.
Text-to-sign model	Reconstructs or selects the sign image corresponding to the predicted word.
Evaluation	Compares predicted text with reference labels and calculates the BLEU score.

4.3 Main Modules

4.3.1 Upload Sign Language Dataset

The user selects the dataset directory through the GUI. The application reads the labelled image folders and prepares the associated words and sign images. The source reports 527 images distributed across 18 word classes. The dataset-loading function also checks for previously saved arrays before processing the image folders.

4.3.2 Preprocess Dataset

The preprocessing module prepares text and sign-image data for model input. The documented operations include shuffling and normalisation. The code excerpt shows

text vectorisation using TfidfVectorizer, image resizing to 128 × 128, conversion to floating-point values and scaling of image values. Landmark features are also standardised using StandardScaler.

4.3.3 Train and Test Split

The processed data is divided into training and testing portions. The project uses 80% of the data for training and 20% for testing. The split allows the model to be evaluated using samples that were held out from the training portion.

4.3.4 Train/Load SignNet II Model

The model module prepares the sign-to-text encoder and text-to-sign image model. The code excerpt defines convolution, pooling, flattening, repeat-vector, attention, LSTM and dense layers for the encoder. It also defines a convolutional encoder-decoder image model. Model weights are loaded from the model directory when available; the documented code includes a checkpoint-based training path for the image model.

4.3.5 Sign to Text to Sign from Video

The user selects a test video. The application reads its frames, detects hand landmarks, preprocesses the landmarks and supplies them to the sign-to-text model. The predicted word is displayed, and the corresponding sign image is shown as the reverse translation output.

4.3.6 Sign to Text to Sign from Webcam

The webcam module opens the camera and processes frames continuously while the capture loop is active. When a hand is detected, the landmarks are extracted and passed to the recognition model. The predicted word is displayed and the corresponding sign image is generated or retrieved for display. The source notes that the user must perform the sign accurately according to the dataset for a correct prediction.

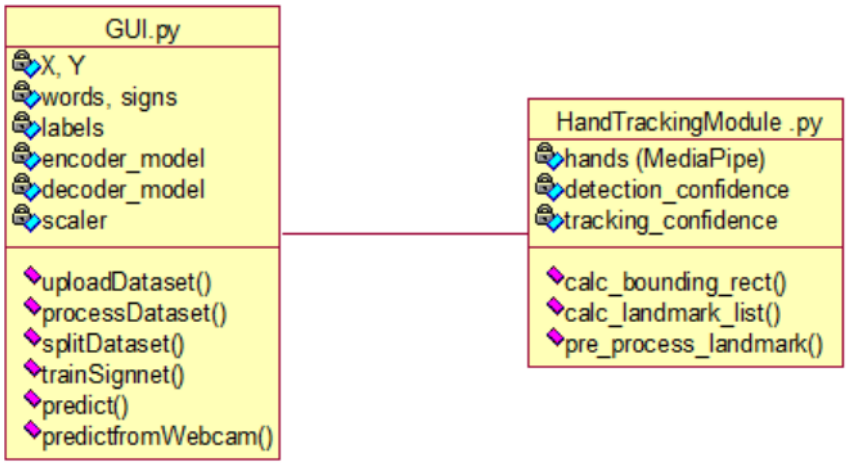


Figure 4.2: SignNet II Class Diagram.

4.4 Core Workflow

The SignNet II system follows the workflow described below:

1. **Start the Application:** Launch the SignNet II application using the Tkinter graphical user interface.
2. **Load the Dataset:** Select and load the sign language dataset containing the predefined ASL sign classes.
3. **Dataset Preprocessing:** Prepare the features, shuffle the samples and normalise the data to ensure consistency during model processing.
4. **Dataset Splitting:** Divide the processed dataset into 80% training data and 20% testing data.
5. **Model Loading or Training:** Load or train the SignNet II model components required for sign recognition and sign-image generation.
6. **Select Input Mode:** Choose either uploaded-video prediction or live-webcam prediction.
7. **Hand Landmark Extraction:** Extract hand landmarks from the visual input and preprocess the feature coordinates for model processing.
8. **Text Prediction:** Process the extracted features through the recognition model to predict the corresponding text word.

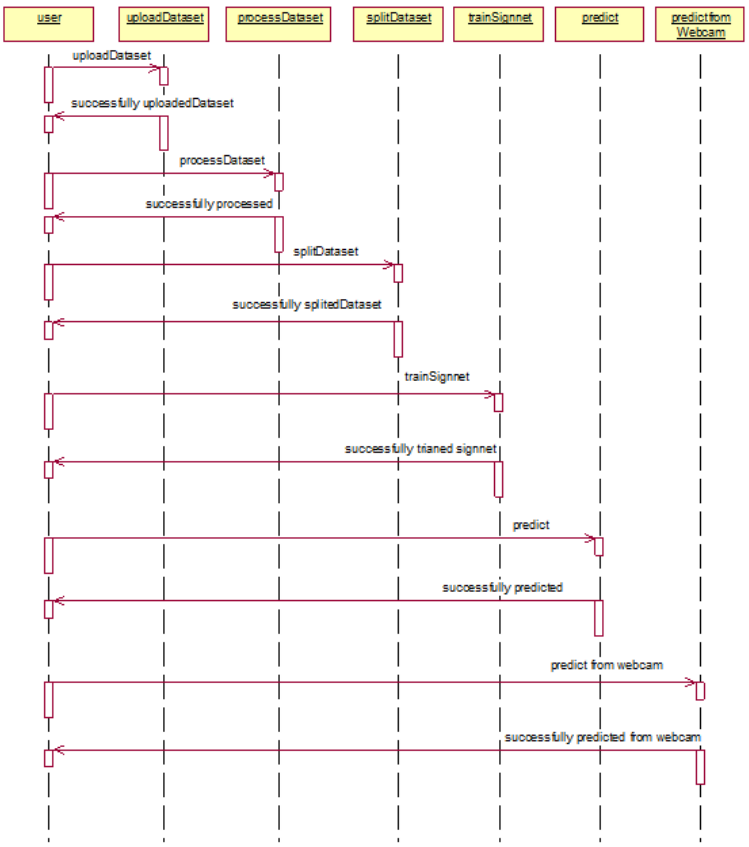


Figure 4.3: User Interaction Sequence Diagram.

Overview:

The image displays a sequence diagram illustrating the step-by-step workflow between a user and various system components.

Key Components:

The lifeline participants include the user, upload Dataset, process Dataset, split Dataset, train Signnet, predict, and predict From Webcam.

Workflow:

The process begins with the user uploading and processing a dataset, followed by splitting the data, training the Signnet model, and finally running predictions either generally or directly from a webcam.

Data Flow Diagram :

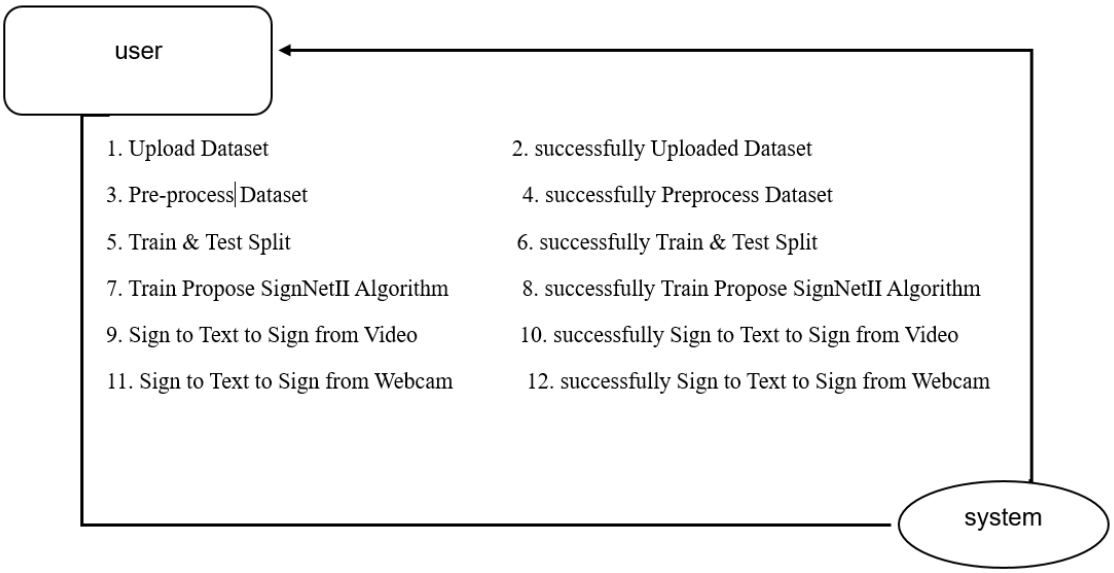


Figure 4.4: SignNet II Data Flow Diagram.

4.5 Logical Data Design

The source project does not describe a relational database or a persistent database schema. The application uses dataset folders, in-memory arrays and saved model-related files. The following logical data groups describe the information required by the processing workflow.

Logical data group	Example contents / purpose
Sign image dataset	Grayscale sign images organised by word-class folders.
Word labels	The 18 supported ASL word labels associated with the image classes.
Landmark features	Hand landmark coordinate pairs, transformed into a flattened and normalised feature vector.
Processed image data	Resized sign images represented as numerical arrays.
Training/testing data	Feature arrays and labels separated into 80% training and 20% testing portions.
Model artefacts	Encoder and decoder weights, saved arrays and model history files.

4.6 Risk Assessment and Remediation Flow

A risk is identified when a workflow stage cannot produce the expected input or output. The proposed response is to stop the dependent operation, display a useful message, correct the input or configuration, and repeat the relevant test.

Workflow stage	Potential issue	Remediation approach
Dataset loading	Folder unavailable or expected data missing.	Verify the selected folder and required class-folder structure before preprocessing.
Preprocessing	Invalid image or feature array.	Check that images and feature arrays can be read and have expected dimensions.
Train/test split	Insufficient or inconsistent samples.	Verify dataset loading and labels before splitting.
Model loading	Weight file unavailable or incompatible.	Check the model directory and ensure matching model files are present.
Video prediction	Video cannot be opened or no hand is detected.	Select a readable video and ensure the hand is visible in the frames.
Webcam prediction	Camera cannot be accessed or hand landmarks are absent.	Check camera availability and lighting/hand visibility; stop capture safely if unavailable.
Text-to-sign output	Predicted class does not map to an available sign image.	Verify the label-to-image mapping and dataset class ordering.

The table presents the potential risks and corresponding remediation approaches associated with different stages of the SignNet II workflow. It identifies possible issues during dataset loading, preprocessing, train-test splitting, model loading, video prediction, webcam prediction and text-to-sign output generation. Each issue is accompanied by a suitable corrective measure, such as verifying dataset folder

structures, checking image and feature dimensions, ensuring model file compatibility, confirming camera availability and validating the mapping between predicted labels and sign images. These measures help minimise processing errors, maintain system reliability and ensure the smooth execution of the application.

4.7 Security and Deployment Options

The uploaded project document describes a local Python desktop application using Tkinter, webcam/video input and locally stored model files. It does not document user authentication, encryption, network services, role-based access control, cloud hosting or a formal security architecture. Such controls should therefore not be represented as implemented features.

The deployment model supported by the source is local execution on a compatible Windows computer with Python and the required libraries installed. The project is launched through a run.bat file. Model files and dataset resources are expected to be available in the local project directory. As a practical precaution, access to the dataset and model files should be limited to trusted users, and unnecessary copies of camera recordings should not be retained.

4.8 Summary

This chapter described the system architecture, six main application modules, end-to-end workflow and logical data groups. It also outlined a risk-remediation flow and the documented local deployment approach. The system is organised around the two translation directions and the GUI that connects them.

Furthermore, the chapter explained how the different modules interact to perform dataset processing, hand landmark extraction, gesture recognition and sign-image generation. Each module has a specific responsibility, ensuring proper coordination between the input, processing and output stages. The architecture provides a structured approach to integrating the machine-learning components with the graphical user interface.

CHAPTER 5

INPUT AND OUTPUT DESIGN

5.1 Introduction

Input and output design defines how information enters the SignNet II application and how the system presents its results. The main inputs are the dataset folder, video frames and live webcam frames. The principal outputs are predicted text, corresponding sign images and the BLEU evaluation score.

5.2 Input Design

5.2.1 Dataset Input

The user selects the dataset directory through the folder-selection dialog. The application reads image files from the labelled class folders and associates each image with its word label. The source describes 18 classes and reports 527 dataset images.

5.2.2 Video Input

For video-based prediction, the user selects a test video through a file-selection dialog. The application reads the video frame by frame and processes the frames for hand landmark detection.

5.2.3 Webcam Input

For live prediction, the application opens the default webcam. Frames are captured and passed to the hand-detection and prediction pipeline. The user performs one of the signs represented in the dataset.

5.2.4 Model and Feature Input

The recognition pipeline receives preprocessed landmark coordinates. The reverse translation path receives the predicted word after it has been converted into the numerical form expected by the text-to-sign component.

5.3 Input Validation

The source document does not provide a complete formal validation specification. The following checks are appropriate to the documented input types and should be applied or verified during operation:

Confirm that the selected dataset directory exists and contains readable image files.
Confirm that image files can be opened and resized to the expected dimensions.
Confirm that each dataset sample is associated with one of the 18 supported labels.
Check that a selected video can be opened and that frames can be read.
Check that the webcam can be opened before starting live prediction.
Check that hand landmarks are present feature data to the recognition model.
Check that processed feature arrays have the expected shape and numerical format.
Confirm that the predicted class index maps to a valid word label and sign image.
These checks are presented as validation requirements for a reliable workflow; the source does not establish that every check is implemented as a dedicated validation message in the GUI.

5.4 Output Design

5.4.1 Text Output

The predicted word is displayed in the Tkinter text area and can also be overlaid on the processed video frame. The source screenshots show example predictions including “can”, “I” and “your”.

5.4.2 Sign Image Output

After a word is predicted, the corresponding sign image is displayed in a separate output window. This provides the reverse translation result and completes the two-way workflow.

5.4.3 Dataset and Processing Output

The application displays the selected dataset path, the number of images found, the available sign labels and a graph showing the image count for each class. It also displays a message when shuffling, vectorisation and normalisation have been completed.

5.4.4 Training and Evaluation Output

The application reports the number of training and testing samples and displays the BLEU score calculated from the predicted and reference word sequences.

5.5 Output Integrity

Output integrity depends on maintaining a correct relationship between the predicted class index, the word label and the corresponding sign image. The same class ordering must be used throughout dataset preparation, model prediction and image lookup. The test and training data should also remain separated according to the stated split so that evaluation is performed on held-out samples.

The BLEU score should be interpreted as the text-sequence comparison metric used in the project. It does not, by itself, establish the visual quality of generated sign images, the usability of the GUI or the accuracy of every possible sign outside the 18-class dataset.

5.6 Summary

This chapter described the dataset, video and webcam inputs and the text, image and evaluation outputs. It also identified validation checks and output-integrity considerations needed to preserve the correct mapping between hand signs, predicted words and sign images.

Additionally, the chapter explained how the input data is processed and converted into meaningful outputs through the system's different modules. Proper validation of input formats, hand landmarks and prediction results helps maintain consistency and reduce errors during translation. These considerations contribute to the overall reliability, accuracy and usability of the SignNet II application.

Additionally, the chapter explained how input data is processed and converted into meaningful outputs through the different modules of the system. Proper validation of input formats, hand landmarks and prediction results helps maintain consistency and reduce errors during translation. The chapter also highlighted the importance of maintaining a clear relationship between the recognised text and its corresponding sign-image representation.

CHAPTER 6

SOFTWARE ENVIRONMENT

6.1 Introduction

The software environment consists of the programming language, GUI framework, computer-vision libraries, numerical-processing tools, machine-learning components and evaluation utilities used by SignNet II. The source project is implemented as a Python desktop application.

6.2 Frontend Environment

The graphical user interface is developed using Tkinter, Python's standard GUI toolkit. The interface contains buttons for dataset upload, preprocessing, train-test splitting, model training and prediction from video or webcam. A text widget and output windows are used to display messages, predicted words and sign images.

The interface is designed as a local desktop application rather than a browser-based frontend. The source code sets the main window title to identify SignNet II and defines controls for the six principal operations.

6.3 Backend Environment

Python coordinates dataset loading, image processing, feature preparation, model execution and prediction. The code imports NumPy for numerical arrays, OpenCV (cv2) for image/video handling, pickle for serialising model history, and operating-system utilities for checking and reading local files.

The source lists Windows 10 as the minimum operating system and Python as the programming language. It also lists minimum hardware requirements of an Intel i3 processor, 1.1 GHz speed, 4 GB RAM and 500 GB hard disk.

6.4 Database and Optional Vector Layer

The source document does not identify a database management system or a vector database. The application stores dataset information in folders and NumPy array files and stores model weights in local files. Text is represented numerically through the vectorisation step shown in the implementation. Therefore, a database or optional vector-search layer is not included in the documented system architecture.

6.5 Security and DevOps

The documented deployment process is local execution through the project's run.bat file. The source does not describe a cloud deployment pipeline, containerisation, automated build system, authentication service or network-based API. Model weights and dataset files are accessed from the local project directory.

For responsible local operation, project files should be kept in a controlled directory, trusted dataset sources should be used, and webcam/video input should be accessed only with the user's knowledge. These are operational recommendations; the source does not claim that a dedicated security module has been implemented.

6.6 Testing Tools

Tkinter interface controls are used to invoke the project functions and inspect visible messages.

OpenCV is used to read video frames, access the webcam and display processed frames.

NumPy and scikit-learn utilities are used in preprocessing and dataset splitting.

Keras model components are used to load and execute the neural-network models.

NLTK sentence_bleu is used to calculate the documented BLEU evaluation score.

6.7 Summary

This chapter presented the Python and Tkinter application environment and the main computer-vision, numerical-processing and machine-learning libraries documented in the source. It also clarified that the project uses local dataset and model files rather than a documented database or cloud deployment platform.

The chapter also highlighted the role of these technologies in supporting the major functions of SignNet II, including dataset preprocessing, hand landmark extraction, gesture recognition, text processing and sign-image generation. The integration of these tools provides the necessary software environment for developing and executing the bidirectional translation workflow through a graphical user interface.

CHAPTER 7

SYSTEM TESTING AND RESULTS DISCUSSION

7.1 Introduction

Testing is performed to verify that individual modules operate correctly and that the complete application supports the intended two-way translation workflow. The source project describes module testing, integration testing, system testing and acceptance testing. It also includes a test-case table and screenshots showing dataset loading, preprocessing, splitting, model evaluation, video prediction and webcam prediction.

7.2 Testing Objectives

Verify that the dataset folder can be selected and its images can be loaded.
Verify that preprocessing and normalisation complete before model execution.
Verify that the training and testing portions are created using the 80:20 split.
Verify that model components and required weights can be loaded or trained.
Verify that video frames and webcam frames can be processed.
Verify that the system displays the predicted word and corresponding sign image.
Verify that the BLEU score is calculated and displayed for the test predictions.

7.3 Testing Methods and Tools

7.3.1 Module Testing

Each module is tested independently so that errors can be identified and corrected without affecting unrelated modules. The source describes testing the dataset, preprocessing, splitting, model and prediction functions separately.

7.3.2 Integration Testing

After individual modules are tested, they are connected and tested as a complete workflow. Integration testing checks whether the output of one stage is accepted correctly by the next stage, such as whether extracted landmarks can be passed to the encoder and whether predicted text can be mapped to a sign image.

7.3.3 System Testing

System testing evaluates the complete application using the expected sequence of operations. It checks the interaction between the GUI, dataset files, model components, video processing and output display.

7.4 Representative Test Cases

The following table consolidates the test scenarios documented in the project and adds clear expected outcomes for reporting.The actual status below is described only where the source provides visible evidence.

ID	Test case	Expected result	Evidence/status in source
TC-01	Upload sign language dataset	Dataset folder is selected and images/classes are read.	Demonstrated in screenshots; 527 images across 18 classes reported.
TC-02	Preprocess dataset	Features are shuffled, vectorised and normalised.	Screenshot narrative reports preprocessing/normalisation completed.
TC-03	Train-test split	Data is divided into 80% training and 20% testing.	The split is specified and displayed in the application workflow.
TC-04	Train/load SignNet II model	Required model components are loaded or training is performed; evaluation is displayed.	The source reports a BLEU result after model execution.
TC-05	Sign-to-text-to-sign from video	Video frames are processed; predicted word and corresponding sign image are displayed.	Screenshots show example predicted words and sign-image output.
TC-06	Sign-to-text-to-sign from webcam	Webcam frames are processed; predicted word and corresponding sign image are displayed.	The source describes a webcam example with predicted word “your” and corresponding image.

The following figures illustrate the main graphical user interface of SignNet II and demonstrate the different stages of its operation, including dataset loading, class distribution and real-time bidirectional sign language translation.

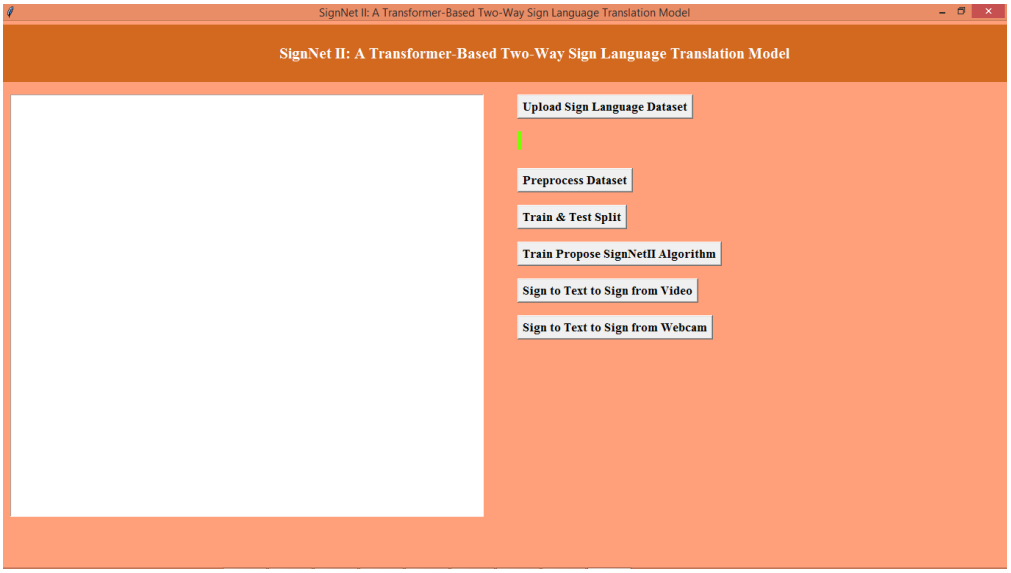


Figure 7.1: SignNet II Main Graphical User Interface

This figure shows the main graphical user interface of the SignNet II application, developed using Python Tkinter. It provides access to the major system functionalities, allowing users to select input options, perform sign recognition and view the generated outputs.

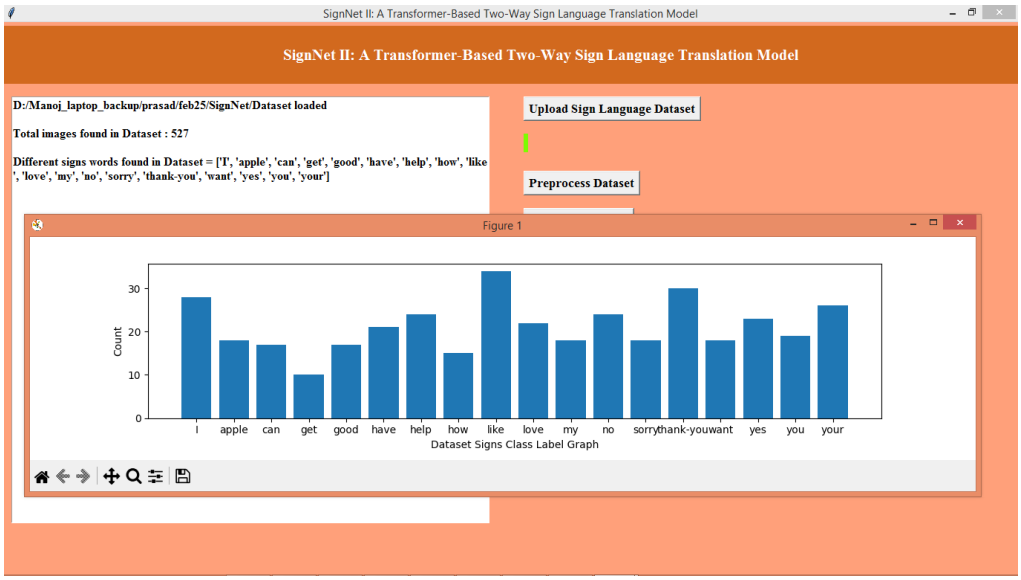


Figure 7.2: Dataset Loading Status and Class Distribution Graph

This figure illustrates the dataset loading process and displays the distribution of images across the 18 ASL word classes. It helps verify that the dataset has been successfully loaded and provides an overview of the available training samples.

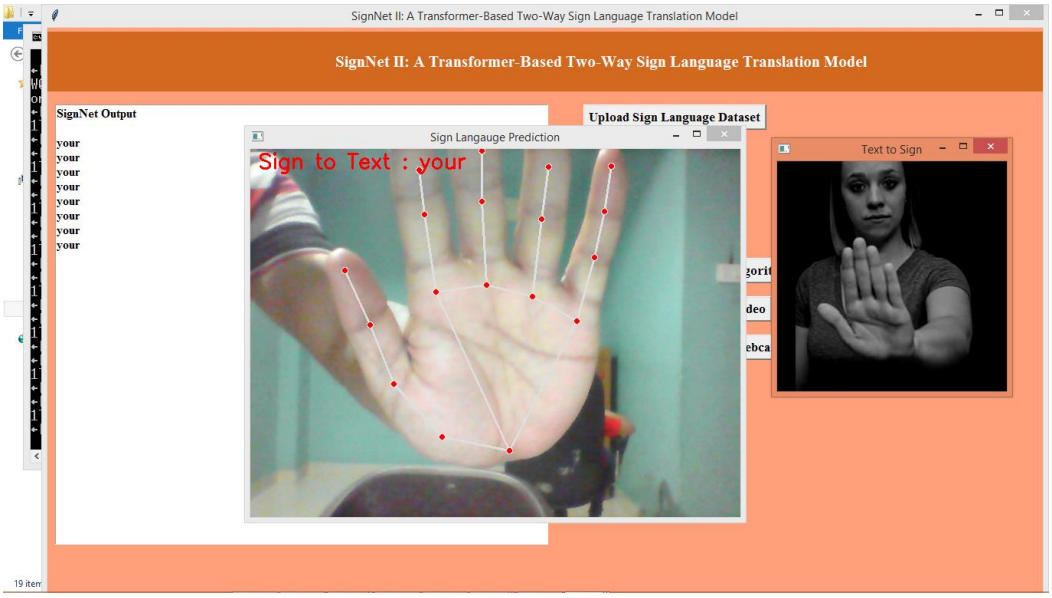


Figure 7.3: Real-Time Webcam Sign-to-Text and Text-to-Sign Translation

This figure demonstrates the real-time webcam-based sign recognition process. The system captures hand gestures, predicts the corresponding text word and displays its associated sign-image representation, illustrating the bidirectional translation functionality of SignNet II.

7.5 Evaluation Metrics

The project uses the BLEU (Bilingual Evaluation Understudy) score to compare predicted text with reference text. In the implementation excerpt, the predicted class labels and true class labels are joined into text sequences. The NLTK sentence_bleu function is then applied, and the resulting value is divided by two before it is displayed as the SignNet II BLEU score.

The source reports a BLEU score of 0.5% for the test data in the model-evaluation screenshot description. This value is reproduced as reported in the project document; it should not be converted into a different scale or interpreted as a general accuracy percentage. BLEU evaluates text correspondence and does not directly measure the quality of the generated sign image.

Although accuracy_score is imported in the code, the source's stated result discussion specifically reports the BLEU score. No separate numerical accuracy, precision, recall, F1-score, latency or image-quality metric is provided in the documented results.

7.6 Results and Evidence

The project document provides the following implementation evidence:

1. **Dataset Loading:** The application successfully loads 527 images belonging to 18 sign-word classes. A class count graph is displayed to illustrate the distribution of images across the different sign categories.
2. **Preprocessing:** The graphical user interface displays the completion of image shuffling, word vectorisation and data normalisation, preparing the dataset for model processing.
3. **Train-Test Split:** The application displays the sizes of the training and testing datasets, demonstrating the division of the processed data according to the 80:20 ratio.
4. **Video Prediction:** The screenshots demonstrate hand key-points overlaid on video frames, along with predicted text labels such as "can" and "I" and their corresponding sign images.
5. **Webcam Prediction:** The documented screenshot shows a hand gesture being recognised as "your" through webcam input, with the corresponding text-to-sign image displayed by the application.
6. **Overall Implementation:** The available evidence demonstrates that the application workflow was exercised across dataset loading, preprocessing, data splitting, model evaluation and both video-based and webcam-based prediction modes. The documented evaluation score and prediction examples should be interpreted within the limitations of the 18-word dataset and the test conditions described in the project.
7. **Graphical User Interface:** The Tkinter-based interface provides a common platform for accessing the different functionalities of SignNet II. It allows users to select input modes, load datasets, initiate prediction operations and view the generated results. This integration makes the application easier to operate without requiring direct interaction with the underlying model code.

8. **Bidirectional Translation:** The implementation demonstrates the integration of sign-to-text recognition and text-to-sign image generation within a single application. The recognised word is presented as text along with its corresponding sign image, providing visual feedback and supporting the project's objective of bidirectional communication.
9. **Testing and Validation:** The documented screenshots and outputs provide evidence of the execution of the major application functions. They help verify the expected behaviour of dataset processing, prediction operations and output generation. However, broader testing across different users, lighting conditions and camera environments would be required to establish the system's overall robustness.

7.7 Limitations

The SignNet II project has certain limitations that define its current capabilities and areas for future improvement. These limitations are described below:

1. **Vocabulary Limitation:** The dataset contains only 18 distinct ASL words and does not cover the complete range of sign language vocabulary required for general communication.
2. **Word-Level Scope:** The project focuses on recognising isolated word-level signs rather than supporting continuous sign language recognition or sentence-level translation.
3. **Input Sensitivity:** Accurate prediction depends on presenting hand signs correctly according to the dataset. Variations in hand positioning and gesture execution may affect recognition results.
4. **Resource Requirements:** Model training and prediction processes are computationally demanding. The source document advises against unnecessarily continuous prediction to avoid excessive resource consumption.
5. **Evaluation Coverage:** The reported BLEU score measures the correspondence between predicted text and reference labels. However, the source does not provide a separate numerical evaluation metric for the quality of generated sign images.

6. **Implementation-Description Differences:** The project description refers to OpenPose for hand key-point extraction and Word2Vec for text representation, whereas the visible implementation excerpts use MediaPipe Hands and TF-IDF for these operations. These differences should be clearly explained during the final technical demonstration.
7. **Limited Robustness Evidence:** The project does not provide extensive evaluation across multiple signers, different backgrounds, lighting conditions and camera viewpoints. Therefore, its performance under varied real-world conditions remains to be established.
8. **Security Documentation:** The project source does not specify the implementation of authentication, data encryption, cloud security or formal privacy controls. These aspects may be considered for future system improvements.

7.8 Summary

This chapter described the testing objectives, testing methods and documented test cases used to evaluate the SignNet II application. The testing covered dataset loading, preprocessing, normalisation, 80:20 train-test splitting, model evaluation and prediction through uploaded video and live webcam input. The documented results include processing 527 images across 18 ASL word classes and a reported BLEU score of 0.5%.

The testing results demonstrate the feasibility of integrating sign-to-text recognition and text-to-sign image generation within a single graphical application. The video and webcam prediction examples show that the system can recognise defined hand gestures and display their corresponding text and sign images.

However, limitations such as restricted vocabulary, input sensitivity, computational requirements and limited evaluation coverage remain. Overall, the testing confirms that SignNet II fulfils its intended functional objectives within the defined project scope and provides a foundation for future improvements in recognition accuracy, vocabulary expansion and real-time performance.

CONCLUSION

The proposed project**, A BIDIRECTIONAL DEEP LEARNING FRAMEWORK FOR REAL-TIME SIGN LANGUAGE TO TEXT AND TEXT TO SIGN TRANSLATION** (SignNet II), aims to improve communication between deaf and hard-of-hearing individuals and others by providing a two-way sign language translation system. The framework is designed to recognize hand gestures from uploaded videos and live webcam input and convert them into corresponding textual representations. It also generates the corresponding sign images from the predicted text, providing a complete communication cycle.

The proposed system uses a word-level American Sign Language (ASL) dataset containing 527 images across 18 different sign classes. The system performs dataset uploading, preprocessing, normalization, and training and testing operations using an 80:20 data split. Hand and finger landmarks are extracted from the input gestures and processed by the recognition model to predict the corresponding words.

The SignNet II application provides a user-friendly graphical interface developed using Python Tkinter. It supports video-based and real-time webcam-based sign language recognition. The predicted text is further processed to generate the corresponding sign image, allowing users to understand the translated output visually. The system evaluates its translation performance using the BLEU score, with the project document reporting a score of 0.5%.

In conclusion, SignNet II presents an approach towards developing an accessible and interactive sign language translation framework. By combining gesture recognition, deep learning models, and visual sign generation, the system demonstrates the potential of bidirectional communication support. Future improvements can focus on expanding the vocabulary, improving recognition accuracy, supporting continuous sentence-level translation, and enhancing real-time processing performance.

Program Outcome Achieved: PO1, PO2, PO3, PO4, PO5.

FUTURE SCOPE

FS1: Expansion of Sign-Language Vocabulary

The current 18-word ASL vocabulary can be expanded by incorporating a larger dataset containing more words and expressions to improve the coverage of the translation system.

FS2: Continuous Sentence-Level Translation

The system can be extended from individual word recognition to continuous sign-language translation, enabling complete sentences and longer sequences to be processed.

FS3: Diverse and Robust Dataset

The dataset can be expanded with samples from different signers, backgrounds, lighting conditions and camera viewpoints to improve the robustness of the model.

FS4: Improved Feature and Temporal Processing

More accurate landmark extraction and improved temporal modelling can be explored to capture fine-grained hand and finger movements more effectively.

FS5: Faster and Efficient Inference

The models can be optimized to reduce processing time and memory consumption, particularly for real-time webcam-based prediction.

FS6: Advanced Transformer and Attention Models

Stronger Transformer architectures and improved attention mechanisms can be incorporated to enhance feature learning and translation performance.

FS7: Comprehensive Performance Evaluation

Future versions can be evaluated using multiple metrics such as accuracy, precision, recall, F1-score and BLEU, along with human assessment for a more comprehensive evaluation.

REFERENCES

[1] N. Shahin and L. Ismail, "ADAT: Novel Time-Series-Aware Adaptive Transformer Architecture for Sign Language Translation," *Scientific Reports*, vol. 16, art. no. 6551, 2026. Nature

[2] R. Damdoo, P. Kumar, and R. Gogoi, "End-to-End Sentence-Level Indian Sign Language Translation with ISH-NEWS Dataset and Transformer Model," *Scientific Reports*, vol. 16, art. no. 32549, 2026. Nature

[3] Z. An, S. Tan, Y. Hwang, K. Nakadai, and R. Kawakami, "SignFLIP: A Unified Model for Sign Language Translation and Generation via Stage-wise Alignment at Scale," *Findings of the Association for Computational Linguistics: EMNLP 2026*, pp. 17562–17576, 2026. ACL Anthology

[4] M. Hu, G. Sun, A. D. Ross, S. Li, and Z. Tao, "Attention-Steered Vision-Language Models for Sign Language Translation," *Findings of the Association for Computational Linguistics: EMNLP 2026*, pp. 30596–30611, 2026. ACL Anthology

[5] Y. Jiang, L. Zhang, X.-Y. Wei, and L. Qing, "Think in Latent Thoughts: A New Paradigm for Gloss-Free Sign Language Translation," *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 9993–10012, 2026. ACL Anthology

[6] S. Gan, X. Liu, Y. Yin, N. Liu, K. Liu, D. Tuerdaken, Z. Jiang, L. Xie, S. Lu, and H. Wen, "Learning Effective Sign Features without Text for Gloss-Free Sign Language Translation," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9827–9836, 2026. CVPR

[7] C. Han, W. Ma, X. Tang, K. Hu, S. Wen, and Y. Xiang, "BoostSLT: Boosting Sign Language Translation via a Plug-and-Play Diffusion-Based Semantic Enhancer," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 28828–28837, 2026. CVPR

[8] E. Colonna, I. Rinaldi, D. Landi, G. Vessio, and G. Castellano, "Handscribe: A Gloss-Free Framework for Sign Language Translation and Gloss Sequence Generation," *Computer Vision and Image Understanding*, vol. 265, art. no. 104674, 2026. ScienceDirect

[9] W. F. Maia, A. M. Lopes, and S. A. David, "Automatic Sign Language to Text Translation Using MediaPipe and Transformer Architectures," *Neurocomputing*, vol. 642, art. no. 130421, 2025. ScienceDirect

[10] Y. Jang, H. Raajesh, L. Momeni, G. Varol, and A. Zisserman, "Lost in Translation, Found in Context: Sign Language Translation with Contextual Cues," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8742–8752, 2025. CVPR

[11] R. Wong, N. C. Camgoz, and R. Bowden, “SignRep: Enhancing Self-Supervised Sign Representations,” *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 22804–22814, 2025. CVPR

[12] J. Kim, H. Jeon, J. Bae, and H. Y. Kim, “Leveraging the Power of MLLMs for Gloss-Free Sign Language Translation,” *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 21048–21058, 2025. CVPR

[13] Y. Li, X. Chen, H. Li, X. Pu, P. Jin, and Y. Ren, “VSNet: Focusing on the Linguistic Characteristics of Sign Language,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 24320–24330, 2025. CVPR

[14] P. A. Dal Bianco, O. A. Stanchi, F. M. Quiroga, and F. Ronchetti, “Multilingual Sign Language Translation with Unified Datasets and Pose-Based Transformers,” *Proceedings of the Workshop on Sign Language Processing*, pp. 27–32, 2025. ACL Anthology

[15] R. Damdoo and P. Kumar, “Enhanced Sign Language Translation Using Three Stream Multimodal Occlusion Resilient Transformer,” *Discover Computing*, vol. 28, art. no. 260, 2025.